

1 CUDA: Parallelization of the Laplace code

We will modify the `laplace0.c` file and `run.sh` to perform our parallelization. Lets review how we have completed the *To Do's* in order to parallelize using CUDA. First, let us look into the `dev_laplace_error`.

```
1 void laplace_step ( float *in, float *out, int n, int m )
2 {
3     __global__ void dev_laplace_error(float* A, float* Anew, float* error, int n, int m) {
4         // Set indices
5         // Set to + 1 because computation starts at index 1
6         int i = blockIdx.x * blockDim.x + threadIdx.x + 1;
7         int j = blockIdx.y * blockDim.y + threadIdx.y + 1;
8
9         if (i >= n - 1 || j >= m - 1) {
10             return;
11         }
12         int idx = i*m + j;
13
14         //Calculate the new value for each element based on the current values of its
15         //neighbors.
16         Anew[idx] = 0.25f*(A[idx-m]+A[idx+m]+A[idx-1]+A[idx+1]);
17
18         //Compute error = maximum of the absolute value
19         float local_error = fabs(Anew[idx]-A[idx]);
20         atomicMax(error, local_error);
21     }
22 }
```

Note, first the indices have been set in order to avoid stepping into the border. Then we have performed our Laplace update. Finally we have updated the error using the `atomicMax` function that computes the maximum of the device threads. Lets look into this function now

```
1 static inline __device__ float atomicMax(float *addr, float val) {
2     unsigned int old = __float_as_uint(*addr), assumed;
3     do {
4         assumed = old;
5         if (__uint_as_float(old) >= val)
6             break;
7
8         old = atomicCAS((unsigned int *)addr, assumed, __float_as_uint(val));
9     } while (assumed != old);
10
11     return __uint_as_float(old);
12 }
```

After that, we calculate the dimensions of the parallelization using a `ceil`.

```
1 int n_matrix_to_compute = n - 2;
2 int m_matrix_to_compute = m - 2;
3 dim3 gridDim((n_matrix_to_compute + THREADS_BLOCK - 1) / THREADS_BLOCK,
4             (m_matrix_to_compute + THREADS_BLOCK - 1) / THREADS_BLOCK);
5 dim3 blockDim(THREADS_BLOCK, THREADS_BLOCK);
6 }
7 }
```

Finally, we enter into the main loop that performs the device (GPU) and host (CPU) calculations:

```

1  int iter = 0;
2  while (error > tol && iter < iter_max) {
3      iter++;
4
5      cudaMemset(error_dev, 0, sizeof(float));
6      // Fill parameters
7      dev_laplace_error<<<gridDim, blockDim>>>(A_dev, Anew_dev, error_dev, n, m);
8
9      cudaMemcpy(&error, error_dev, sizeof(float), cudaMemcpyDeviceToHost);
10
11     error = sqrtf(error);
12
13     float *swap = A_dev;
14     A_dev = Anew_dev;
15     Anew_dev = swap; // swap pointers A_dev & Anew_dev
16
17     if (iter % (iter_max / 10) == 0)
18         printf("%5d, %.6f\n", iter, error);
19 }
20
21 cudaMemcpy(A, A_dev, n * m * sizeof(float), cudaMemcpyDeviceToHost);

```

Note that we also allocate memory into the device. Then, GPU calculations are done in the Kernel region given by the `dev_laplace_error`. After that, we copy the `error_dev` into the host. This creates one of the most important **bottlenecks** in our program: communication between device and host. This is heavily limited by the PCI express memory that works hard to move the data from GPU to CPU. We will see in the profiling how it takes the great majority of time. Finally, after exiting the loop, we copy the information of the final matrix into the host. It is important to free the variables created in the device with a `cudaFree`.

2 Performance Analysis

Table 1: Comparing execution times (CUDA vs OpenACC)

Configuration	Block Size	Threads/Block	Time (s)	Observations
CUDA Test 1	8×8	64	12.5	Low use of SM (block limit)
CUDA Test 2	16×16	256	10.0	Best performance
CUDA Test 3	32×32	1024	19.8	Low use of SM (block size limit)
CUDA Test 4	64×64	4096	Error	Exceeding maximum threads per block
<i>OpenACC</i>	-	-	<i>3.6</i>	Reference with OpenACC

Benchmarking across three different block dimensions (8×8 , 16×16 , and 32×32) yielded execution times of 12.5s, 10.0s, and 19.8s, respectively. While the 16×16 configuration proved optimal, a comparison with the OpenACC implementation (which executed in only 3.6s) highlights significant room for optimization. Notably, a 64×64 configuration failed because it exceeds the hardware limit of 1024 threads per block.

The superior performance of the 16×16 configuration can be explained by GPU architecture and occupancy. A Stream Multiprocessor (SM) typically supports a maximum of 8 resident blocks and 1536 threads. While all tested sizes are multiples of the warp size (32 threads), avoiding warp divergence—they differ in how they saturate the SM.

With a 32×32 block (1024 threads), only one block can fit per SM (since two would exceed the 1536-thread limit), resulting in a theoretical occupancy of only 66% ($1024/1536$). Conversely, with tiny 8×8 blocks (64 threads), even if the SM fills its maximum slot of 8 blocks, only 512 threads are active ($8 * 64$), leading to poor utilization. The 16×16 size strikes the best balance, maximizing thread occupancy within the SM constraints.

Regarding memory overhead, data transfer between the CPU and GPU is a major bottleneck due to the PCIe bus, which is limited by bandwidth (approx. 64 GB/s) and high latency. Therefore, minimizing host-to-device transfers is critical. Furthermore, on-chip memory optimization is essential. By

implementing tiling, we can leverage Shared Memory, which is significantly faster than Global Memory. This involves copying a data 'tile' from Global to Shared Memory, performing computations locally, and writing the result back, thereby reducing latency and bandwidth pressure.

Additionally, an optimization using Shared Memory was attempted to improve the error reduction process. The strategy involved computing the maximum error locally within each block to minimize the contention of atomic operations on global memory. However, this approach resulted in performance degradation. The latency introduced by the necessary thread synchronization barriers `__syncthreads()` outweighed the benefits of reducing global memory traffic, effectively stalling the execution pipeline and increasing the total runtime.